LABORATORY MANUAL

Breast Cancer Diagnosis Using Supervised Machine Learning

A Practical Laboratory Exercise on Data Preprocessing, Feature Selection, Model Development, Hyperparameter Optimization, and Performance Evaluation

Laboratory Area	Machine Learning / Data Analytics
Problem Type	Binary Classification
Dataset	Breast Cancer Wisconsin (Diagnostic) Dataset
Target Classes	Benign (B) and Malignant (M)
Recommended Environment	Python with Jupyter Notebook or Google Colab

Experimental Note: The workflow in this laboratory manual follows the general methodology of the reference study while applying a leakage-safe experimental order in which the train-test split is performed before preprocessing parameters, feature selection, and model optimization are fitted.

Paper did it afterwards

1. Laboratory Objectives

- Prepare a real-world breast cancer classification dataset for supervised machine learning.
- Perform a train-test split before fitting preprocessing and feature-selection procedures.
- Apply data preprocessing without introducing information leakage from the test set.
- Conduct exploratory data analysis to understand distributions, class composition, and feature relationships.
- Apply and compare multiple feature-selection techniques.
- Train and compare several supervised machine-learning classifiers.
- Perform hyperparameter optimization using cross-validation on the training data.
- Evaluate classification models using complementary performance metrics and visualizations.
- Identify the most effective combination of feature-selection technique and classifier.

2. Dataset

Use the Breast Cancer Wisconsin (Diagnostic) Dataset available from Kaggle. The dataset contains diagnostic measurements used to classify breast tumors as benign or malignant.

Dataset download location: Kaggle – Breast Cancer Wisconsin Data

Dataset URL: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data

- Approximately 569 patient records.
- Thirty diagnostic input features.
- An identification column that should not be used as a predictive feature.
- A diagnosis target representing benign and malignant cases.

3. Required Experimental Workflow

Step	Activity
1	Load the dataset
2	Inspect data quality and structure
3	Define input features and target
4	Perform stratified train-test split
5	Handle missing values using training-data parameters
6	Scale features using training-data parameters
7	Conduct exploratory data analysis
8	Apply feature-selection methods on training data
9	Construct corresponding training and test feature subsets
10	Train supervised machine-learning models
11	Perform hyperparameter optimization using training data
12	Generate predictions on the untouched test data
13	Evaluate, compare, and select the best model

4. Step 1: Load the Dataset

Laboratory Tasks

- Read the CSV file into a DataFrame.
- Display the first few records.
- Inspect dataset dimensions, column names, and data types.

Why this step is performed: This provides an initial understanding of the dataset structure before any transformation is performed.

***Possible functions/methods**: pd.read_csv(), .head(), .shape, .columns, .info()*

5. Step 2: Inspect Data Quality

Laboratory Tasks

.columns.tolist()

- Check for missing values.
- Check for duplicate records.
- Generate descriptive statistics.
- Identify unnecessary or non-predictive columns.

Why this step is performed: Data-quality issues may affect model training and can produce misleading experimental results.

***Possible functions/methods**: .isnull(), .sum(), .duplicated(), .drop_duplicates(), .describe()*

6. Step 3: Define Features and Target

Laboratory Tasks

- Remove the identification column from the predictive input.
- Separate the diagnosis column as the target variable.
- Assign the remaining diagnostic variables as input features.
- Convert the diagnosis labels into numerical form when required.

Why this step is performed: The identification field does not represent a medical characteristic and should not influence prediction.

***Possible functions/methods**: .drop(), .map(), .replace()*

7. Step 4: Perform the Train-Test Split First

Laboratory Tasks

- Divide the dataset into training and testing subsets.
- Use an approximately 80:20 split.
- Apply stratification based on the diagnosis target.
- Set a random state to ensure reproducibility.

Why this step is performed: This is the critical first transformation in the experimental pipeline. The test set must remain unseen during preprocessing, feature selection, and hyperparameter tuning to prevent data leakage.

***Possible functions/methods**: train_test_split()*

8. Step 5: Handle Missing Values

Laboratory Tasks

- Check the training data for missing values.
- Fit the imputation procedure using the training data only.
- Transform the training data.
- Apply the same fitted imputer to the test data.

Why this step is performed: Imputation parameters calculated from the full dataset would allow information from the test set to influence the training pipeline.

***Possible functions/methods**: SimpleImputer(), .fit(), .transform(), .fit_transform()*

9. Step 6: Scale Numerical Features

Laboratory Tasks

- Fit a scaling procedure using the training features only.
- Transform the training features.
- Transform the test features using the same fitted scaler.

Why this step is performed: Scaling places features with different numerical ranges on a comparable scale and is especially important for algorithms sensitive to feature magnitude.

***Possible functions/methods**: MinMaxScaler(), .fit(), .transform(), .fit_transform()*

10. Step 7: Perform Exploratory Data Analysis

Laboratory Tasks

- Examine the class distribution.
- Visualize selected feature distributions.
- Inspect relationships among features.
- Generate a correlation matrix and heatmap.

Why this step is performed: EDA helps identify class composition, unusual distributions, potential outliers, and redundant feature relationships.

***Possible functions/methods**: .value_counts(), sns.countplot(), sns.violinplot(), sns.boxplot(), sns.histplot(), .corr(), sns.heatmap()*

11. Step 8: Correlation-Based Feature Selection

Laboratory Tasks

- Calculate correlations among training features.
- Identify highly correlated feature pairs.
- Select one representative feature or remove redundant features according to a chosen threshold.

Why this step is performed: Highly correlated variables may provide overlapping information. Removing redundancy can simplify the model and reduce multicollinearity.

***Possible functions/methods**: .corr(), np.triu(), .drop()*

12. Step 9: Chi-Square Feature Selection

Laboratory Tasks

- Calculate the relationship between each feature and the target.
- Rank features according to their Chi-Square scores.
- Select the desired number of high-ranking features.

Why this step is performed: This method identifies features that show stronger statistical association with the target class.

***Possible functions/methods**: chi2(), SelectKBest(), .fit(), .transform()*

13. Step 10: Recursive Feature Elimination (RFE)

Laboratory Tasks

- Choose an estimator.
- Fit the estimator using the training data.
- Rank features.
- Iteratively eliminate less important features until the desired subset is obtained.

Why this step is performed: RFE searches for a reduced subset of features that can maintain strong predictive performance.

***Possible functions/methods**: RFE(), .fit(), .support_, .ranking_*

14. Step 11: Recursive Feature Elimination with Cross-Validation (RFECV)

Laboratory Tasks

- Use recursive feature elimination together with cross-validation.
- Allow validation performance to guide the number of retained features.

Why this step is performed: RFECV extends RFE by incorporating validation performance into the feature-subset selection process.

***Possible functions/methods**: RFECV(), .fit(), .support_*

15. Step 12: Random Forest Feature Importance

Laboratory Tasks

- Train a Random Forest classifier using the training data.
- Extract feature-importance scores.
- Rank features and retain an appropriate subset.

Why this step is performed: Tree-based models can estimate the contribution of individual features to the classification process.

***Possible functions/methods**: RandomForestClassifier(), .fit(), .feature_importances_*

16. Step 13: Extra Trees Feature Importance

Laboratory Tasks

- Train an Extra Trees classifier.
- Extract feature-importance scores.
- Rank and select features.

Why this step is performed: This provides another tree-based perspective on feature relevance.

***Possible functions/methods**: ExtraTreesClassifier(), .fit(), .feature_importances_*

17. Step 14: L1-Based Feature Selection

Laboratory Tasks

- Fit an L1-regularized model using training data.
- Identify features with non-zero or sufficiently important coefficients.
- Retain the selected subset.

Why this step is performed: L1 regularization can shrink less useful feature coefficients toward zero and thereby reduce the effective feature set.

***Possible functions/methods**: LogisticRegression(), SelectFromModel(), .coef_*

18. Step 15: Voting-Based Feature Selection

Laboratory Tasks

- Record whether each feature is selected by each feature-selection method.
- Assign one vote for each selection.
- Calculate total votes.
- Rank features according to their total votes.
- Select the most consistently supported features.

Why this step is performed: Different selection methods may emphasize different properties of the data. Voting identifies features that repeatedly appear important across multiple methods.

***Possible functions/methods**: pd.DataFrame(), .sum(), .sort_values()*

19. Step 16: Construct Feature-Selected Datasets

Laboratory Tasks

- For every feature-selection strategy, construct the corresponding training feature matrix.
- Apply exactly the same selected features to the test set.
- Maintain the same feature order in both sets.

Why this step is performed: Feature selection must be determined using training data only, after which the learned selection is applied unchanged to the test data.

***Possible functions/methods**: .transform(), .loc[], .columns, .support_*

20. Step 17: Train Supervised Machine-Learning Models

Laboratory Tasks

- Train Logistic Regression.
- Train Random Forest.
- Train Support Vector Machine.
- Train Extra Trees.
- Train K-Nearest Neighbors.
- Train Decision Tree.
- Train Naïve Bayes.

Why this step is performed: Comparing multiple classifiers helps determine whether a particular feature subset is more suitable for a specific learning algorithm.

***Possible functions/methods**: LogisticRegression(), RandomForestClassifier(), SVC(), ExtraTreesClassifier(), KNeighborsClassifier(), DecisionTreeClassifier(), GaussianNB(), .fit()*

21. Step 18: Hyperparameter Optimization

Laboratory Tasks

- Define candidate hyperparameter values.
- Perform cross-validation using the training data only.

- Identify the best-performing parameter configuration.
- Refit the final model using the selected configuration.

Why this step is performed: Hyperparameter optimization improves the likelihood that the model configuration will generalize well beyond the samples used for fitting.

***Possible functions/methods**: GridSearchCV(), RandomizedSearchCV(), StratifiedKFold()*

22. Step 19: Generate Test Predictions

Laboratory Tasks

- Use the final trained model to predict diagnosis labels for the untouched test set.
- Generate probability or decision scores where required for curve-based evaluation.

Why this step is performed: Test predictions provide an unbiased estimate of final model performance when the test set has not influenced earlier experimental decisions.

***Possible functions/methods**: .predict(), .predict_proba(), .decision_function()*

23. Model Evaluation and Required Results

Every selected model and feature-selection combination should be evaluated using complementary metrics. A single metric such as accuracy should not be used as the sole basis for selecting the final diagnostic model.

Result	Purpose	Possible Function / Method
Accuracy	Measures the overall proportion of correct predictions.	accuracy_score()
Precision	Measures the reliability of positive predictions.	precision_score()
Recall	Measures the ability to correctly identify actual positive cases.	recall_score()
F1-Score	Balances precision and recall.	f1_score()
Classification Report	Summarizes class-wise precision, recall, F1-score, and support.	classification_report()
Confusion Matrix	Shows true/false positive and true/false negative outcomes.	confusion_matrix(), ConfusionMatrixDisplay()
ROC Curve	Shows discrimination performance across thresholds.	roc_curve(), RocCurveDisplay()
ROC-AUC	Summarizes ROC performance in a single score.	roc_auc_score()
Precision-Recall Curve	Useful for examining precision-recall trade-offs.	precision_recall_curve(), PrecisionRecallDisplay()
Feature Importance Plot	Shows influential variables for applicable models.	.feature_importances_, sns.barplot(), plt.bar()

24. Required Visual Outputs

- Class-distribution plot.
- Selected feature-distribution plots.
- Correlation heatmap.
- Feature-selection or feature-importance visualization.
- Confusion matrix for the best-performing model.
- ROC curves for selected competing models.
- Precision-Recall curves for selected competing models.

25. Final Results Table

Prepare a comparison table containing the final test-set performance of each evaluated configuration.

Feature Selection	Model	Accuracy	Precision	Recall	F1	ROC-AUC
Correlation						
Chi-Square						
RFE						
RFECV						
RF Importance						
Extra Trees						
L1-Based						

Feature Selection	Model	Accuracy	Precision	Recall	F1	ROC-AUC
Voting-Based						

26. Final Analysis and Discussion

The laboratory report should discuss the following questions:

- Which feature-selection method produced the most useful subset of features?
- Which classifier achieved the strongest overall performance?
- Did feature selection improve performance compared with using all available features?
- Which model provided the most suitable balance between precision and recall?
- How many false positive and false negative predictions were produced by the final model?
- Considering the diagnostic context, which classification error should receive particular attention and why?
- Did different feature-selection methods favor different machine-learning algorithms?

27. Laboratory Conclusion

The final conclusion should identify the best-performing combination of feature-selection method and supervised machine-learning classifier based on the complete set of evaluation metrics. Students should justify their selection using test-set results rather than relying only on training accuracy or a single metric.

Good experimental practice: The test set should remain untouched until final evaluation. Any imputation parameters, scaling parameters, selected features, and hyperparameter decisions must be learned from the training data and then applied to the test data.